

High Performance Inferencing for LLMs

Ravishankar Ravindran, Ph.D.
(Emerging Open Tech Foundation)

Nov 1, 2025

Abstract

Inferencing has become ubiquitous across cloud, regional, edge, and device environments, powering a wide spectrum of AI use cases spanning vision, language, and traditional machine learning applications. In recent years, Large Language Models (LLMs), initially developed for natural language tasks, have expanded to multimodal applications including vision speech, reasoning and planning each demanding distinct service-level objectives (SLOs). Achieving high-performance inferencing for such diverse workloads requires both model-level and system-level optimizations.

This talk focuses on system-level optimization techniques that maximize token throughput, achieve user experience metrics and inference service-provider efficiency. We review several recent innovations including KV caching, Paged/Flash/Radix Attention, Speculative Decoding, P/D Disaggregation and KV Routing, and explain how these mechanisms enhance performance by reducing latency, memory footprint, and compute overhead. These techniques are implemented in leading open-source inference frameworks such as vLLM, SGLang, Hugging Face TGI, and NVIDIA's Tensor-RT-llm, which form the backbone of large-scale public and private LLM serving platforms.

Attendees will gain a practical understanding of the challenges in delivering scalable, low-latency LLM inference, and of the architectural and algorithmic innovations driving next-generation high-performance inference systems.



Bio

Ravishankar Ravindran has over 24 years of experience contributing to advanced data networking products and research. He is an accomplished Professional currently on an advisory role to eOTF as Technical Director. Previous to this, he was the Telco Architect at F5, he led the system architecture and design for F5's Telco Cloud platform, supporting 5G vRAN and Core workloads. His work included active participation in standards development, particularly in the O-RAN Alliance's Working Group 6 (Cloud Architecture and Orchestration), and contributions to the Nephio project under the Linux Foundation Networking (LFN), focusing on Kubernetes-based domain orchestration for Telco use cases spanning RAN, Core, and Transport networks.

Previously, as Chief Architect at Corning Inc., he focused on the architecture and design of disaggregated RAN (CU/DU) for third-party cloud platforms and contributed to the integration of vDU with third-party O-RUs based on O-RAN's Open Fronthaul (O-FH) specifications, with a focus on the M-plane. Prior to that, he served as Chief Architect at Sterlite Technologies (STL), where he was responsible for the end-to-end design of multi-tier RAN Intelligent Controllers (RICs), aimed at optimizing large-scale RAN systems through xApps such as Mobile Load Balancing, Traffic Steering, and Dynamic Spectrum Sharing.

Before this, he led the Future and Network Theory Lab at Futurewei (Huawei Technologies) as a Principal Researcher, focusing on efficient networking for cloud robotics, autonomous vehicles, and drone systems. His research emphasized next-generation networking requirements, including information-centric networking (ICN), software-defined networking (SDN), and network virtualization—particularly addressing challenges in mobility, content distribution, and content-centric routing protocols.

Prior to this role, he was part of the CTO Office at Nortel, where he was a member of the Advanced Technology Group, working on research areas such as control plane routing protocols for IP/(G)MPLS, L2/L3 Virtualization services, scheduling problems in 4G wireless, and end-to-end QoE/QoS engineering for multimedia services. He later served as a Technology Advisor at Avaya.

Ravindran has been an active contributor to numerous standardization bodies including the IETF, ITU, ATIS, the O-RAN Alliance, and LFN's Nephio. He participated in the ITU's Focus Group on IMT-2020, helping to define early standards for 5G. He holds a Ph.D. in Electrical Engineering from Carleton University, has served as an editor for the Springer Photonic Network Communications (PNET) journal, and has been part of technical program committees for top-tier conferences. Ravindran is a (co-)inventor on over 90 granted and filed U.S. patents (with additional patents pending) and has authored over 50 peer-reviewed papers in IEEE and ACM venues. His research and patents have been cited over 4,000 times, according to his [Google Scholar profile](#).

Agenda

- **Inferencing for LLMs and Performance Objectives**
- **Transformer Complexity**
- **Model versus System-centric Optimization**
- **System Optimization Techniques for LLM Inferencing**
 - KV Caching & Prefix Caching
 - Continuous Batching & Prefix Chunking
 - Paged/Radix/Flash Attention
 - Speculative/Constraint Decoding
 - Prefill/Decode Disaggregation
 - KV Routing and Load Balancing
 - Large Scale Expert Parallelism
- **Inferencing System Architecture**
 - vLLM and SGLang Inferencing Platforms
- **Conclusions**

Inferencing for LLMs

- **Inferencing is when a trained model (Transformer, CNN, ML etc.) are used in production handling new data inputs serving a given use case**
- **Inferencing for LLM's has become a center of focus as Transformers have been proven to handle Language, Vision, Speech tasks quite well, along with Reasoning and Planning tasks.**
- **With this also the SLOs for the end user becomes heterogenous considering the different use cases.**
 - Chatbots, Reasoning/Planning, Image Recognition, Speech Translation, Contact Center etc
- **Inferencing can be distributed :**
 - Inferencing at different layers of the network hierarchy - central/region/edge/device etc.
 - Inferencing system can be scaled across devices (relevant to this talk)
- **Optimized Inferencing systems is important to save costs (user/service provider), Energy usage and meet user objectives**

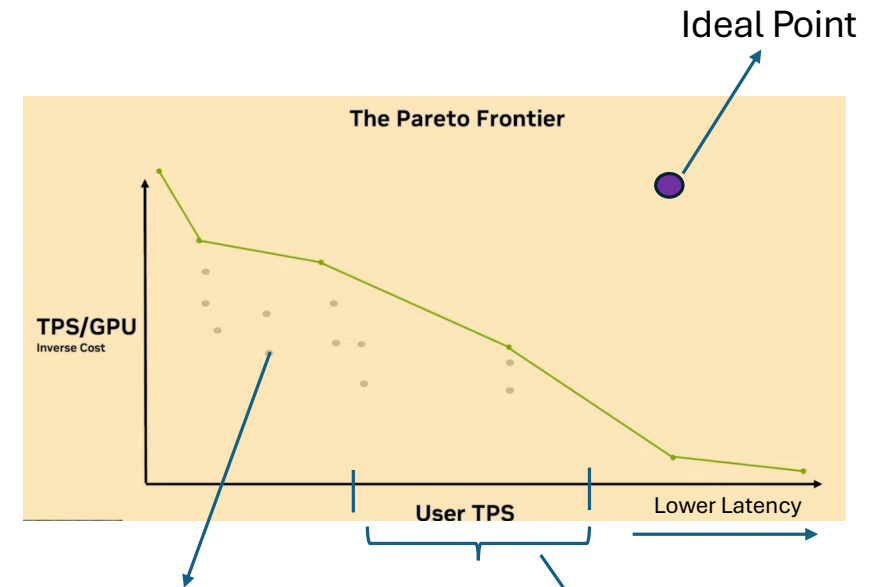
Performance Objectives for LLM Inferencing

- **Performance Metrics**

- *Time for First Token (TFTT)*
 - From an end-user's perspective it is from the moment Prompt is sent to the time the first token is received
- *Inter-token Latency (ITL) & Time per Output Token (TPOT)*
 - The latency between tokens & the average of the ITLs for the latter.
- *Token Throughput*
 - From end-user perspective it is the received tokens/s, and model service provider its aggregate tokens/s for a set of users.
- *Accuracy**:
 - Proportion of instances over a set of query's when the inferencing engine gives a satisfactory response.

- **Optimization Objective**

- The optimization objective is to find an operating point so that that the SLO (user latency) is met, while maximizing system throughput with the given resource and minimize the energy consume
- Pareto frontier for an inferencing system demonstrates a trade-off between user experience and inferencing system's throughput



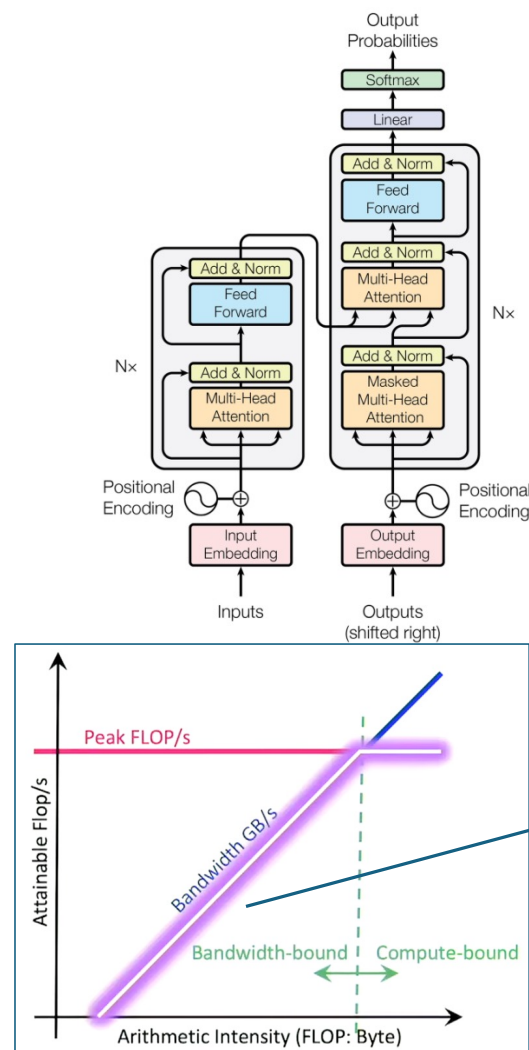
- Different choice of LLM, Batching, scheduling, hyper parameters leads to different operating point considering system constraints

- For desired user latency range e.g. (20-50 tokens/s). Maximize the TPS/GPU (reducing cost/token)

Transformer Complexity

- **These are the Key Transformer parameters that determine the dominant compute and memory complexity**
 - Batch Size : B
 - Sequence Length : N
 - Hidden Dimension : D
 - Number of Layers : M
- **Compute/Memory Complexity**
 - $O(M \cdot B \cdot N^2 \cdot D + M \cdot B \cdot N \cdot D^2)$ & $O(B \cdot M \cdot N^2 + B \cdot M \cdot N \cdot D)$
- **First term is from the Attention operation and the second from Q,K,V Projection.**
 - The Feed Forward complexity is not being considered here
- **For large Context Windows N dominates (Prefill), for smaller context sizes, D dominates (Decode).**
 - Prefill is Compute Bound and Decode is Memory Bound
 - Determined by Arithmetic Intensity (Y) of the GPU
 - $Y = \text{FLOPS} / \text{Memory_Bandwidth (vRAM and the GPU Core)}$
 - If $(\text{Compute} / \text{Byte-Transferred}) \geq Y$ (compute bound)
 - Else Memory Bound
 - E.g. for H100, the $Y \sim 295$
 - Techniques used in the Memory Bound regime:
 - Increase the memory bandwidth (HW), reduce the data that is being transferred, more intelligent Kernel Operation etc.

Transformer Architecture (Vaswani et al, 2017)



- Where you lie depends on the kernel operation
- Most of the inference optimization is focused to improve operation in this memory-bound zone

Model-centric vs System-Centric Optimization

- **Model Centric optimizes the base Transformer model for smaller memory usage , latency, compute efficiency or disaggregation**

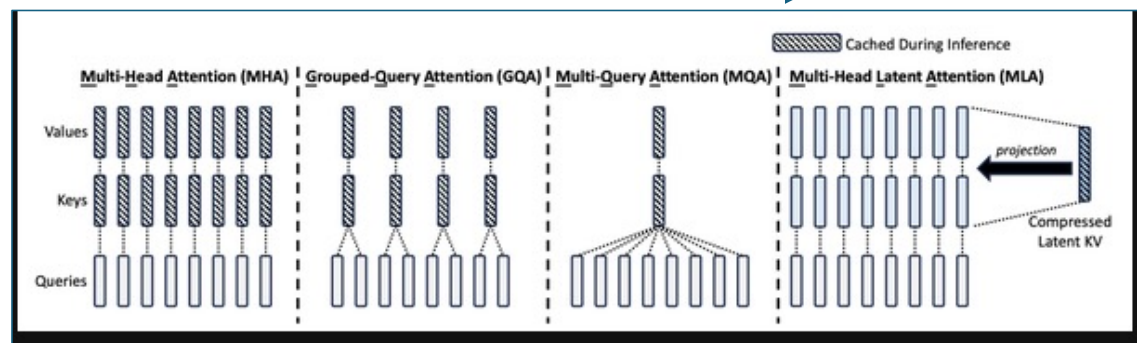
- Accuracy implications
- Mixture-of-Experts
- Group Query Attention
- Multi-head Latent Attention
- Cross-layer/Local Attention
- Quantization (weights, KV Cache, Activation) etc.

- **System-Centric Optimization**

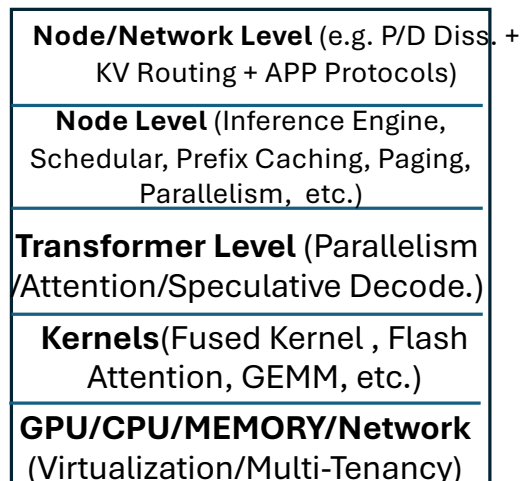
- Generally model-agnostic
- KV Caching
- Continuous Batching
- Chunked Prefill
- Paged/Radix/Flash Attention
- Speculative Decoding
- P/D Disaggregation
- KV Routing

Fig from [1] in Reference.

Reduced KV Cache requirement →



Examples of Model-Centric Optimization



System-centric Optimization for Inferencing

KV Caching

- In a vanilla feed forward case, if there are T tokens to the generated, the total compute is $O(T^3)$
 - Compute Bound, but higher ITL
- KV Caching is used to improve ITL latency by trading of compute for memory
 - Need to transfer the KV cache for each Decode makes it Memory Bound
 - Arithmetic Intensity $\sim O(B \cdot N)$, $N=1$ for Decode, hence B has to be high (may not be feasible)
- In the first pass, the KV cache is saved, that is incrementally used in the decode phase for next token generation
- Since the KV Cache can be 10's of GB ($\sim B \cdot N \cdot \text{\#of Layers} \cdot \text{hidden_dim}$), for a single request, has to be stored in the HBM

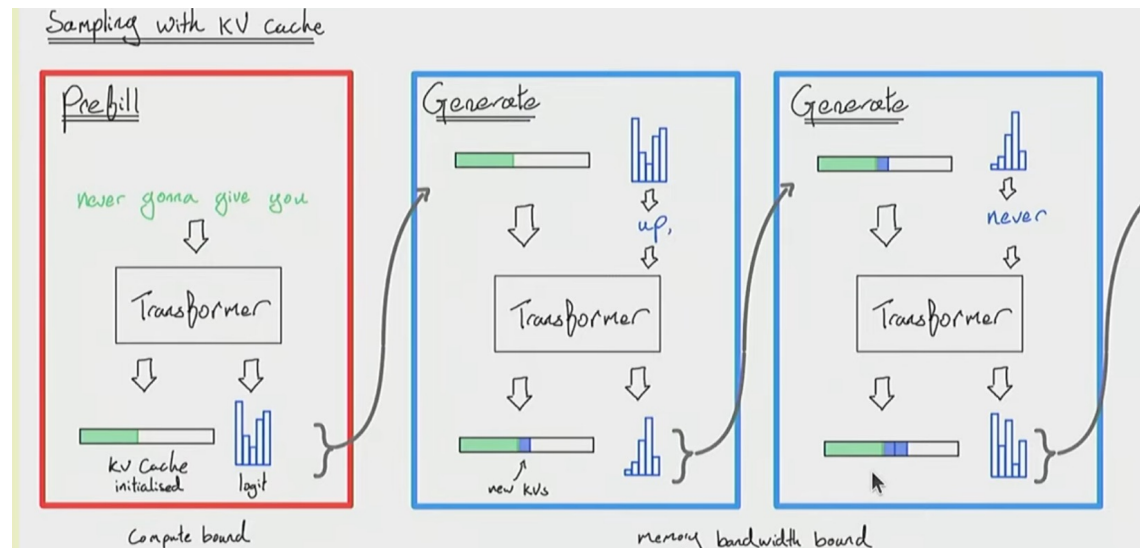


Fig from – CS336, Stanford, Youtube

Continuous Batching and Chunked Prefill

- **Static Batching**

- In a given Batch, wait for the straggler sequence (longest generative request) to finish till you start processing the next request

- **Continuous Batching**

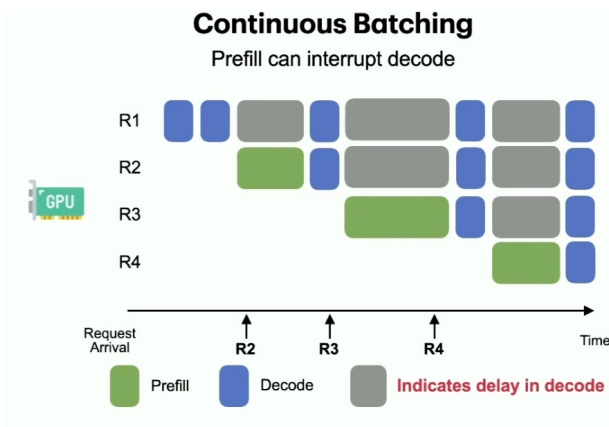
- Batching multiple requests is required to maximize the utilization of the GPU, and also reducing the end user latency
- User requests are random or have different ISL/OSL distributions requires to wait for the whole Batch to finish processing
 - In the e.g. without Continuous Batching, S5/S6/S7 would have to wait till S2 finished generation
- Continuous Batching allows new requests to be inserted current Batch while it is still being processed

- **Chunked Prefill**

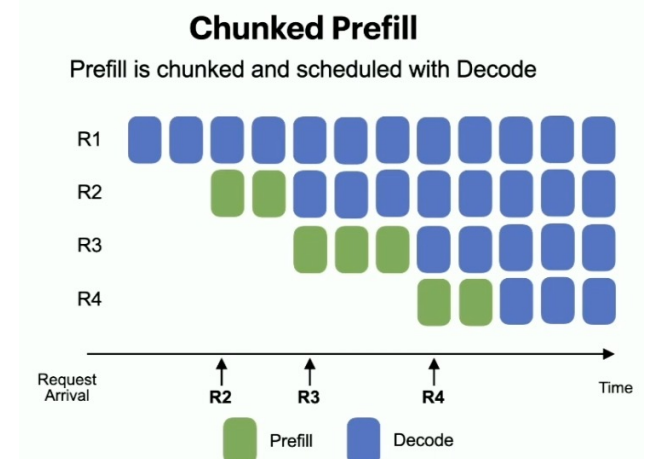
- Continuous Batching could disrupt the Decode generation (because of limited KV Cache memory) hence worsen delay for certain users
 - E.g. R1 Decode generation is severely affected as R2/R3/R4 joins the on going Batch processing
- Chunked Prefill can achieve 2x performance in terms of latency in large Batch size regime, but the TFFT may suffer a bit.

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	END	S_6	S_6
S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	END	S_5	S_5	S_5
S_4	S_4	S_4	S_4	S_4	S_4	END	S_7

Continuous Batching



Continuous Batching Drawback



Chunked Prefill

Paged Attention and Prefix Caching

- **Addresses the memory fragmentation of the GPU resources because of inefficient usage of the KV Cache**
 - Earlier technique was to reserve KV Cache considering the worst case Prompt + Output Token KV Cache/Prompt (e.g. 2k Tokens)
 - Each token can take 800KB-1MB or more
 - $(2(K/V) * \# \text{ of layers} * \text{hidden_dim} * 2)$ (FP-16)
 - Memory for 2K tokens/request $\sim 1.6\text{GB}$
- **Paged Attention**
 - Memory manager maps the tokens to the KV memory blocks of the GPU
 - Logically sequence tokens can map to discontinuous Physical Memory blocks
 - KV Blocks can be reused across prompts and different decoding strategies
 - Parallel Sampling, Beam Search, Prefix Sharing
 - Reuse is at the granularity of a Block, if a Block is only partially filled then a new copy of it is made
- **Prefix Caching**
 - This idea can be extended across multiple prompt generations, when the KV Caches are not evicted after the Decode generation process.
 - The tokens are stored in Block Hash tables, which is used to match the incoming prompts to resolve to the physical KV blocks in the GPU.
 - Also require Cache eviction strategy like LRU to manage the KV Cache

[1] Wooksuk Kwon et al, “Efficient Memory Management for Large Language Model Serving with Paged Attention”, <https://arxiv.org/abs/2309.06180>

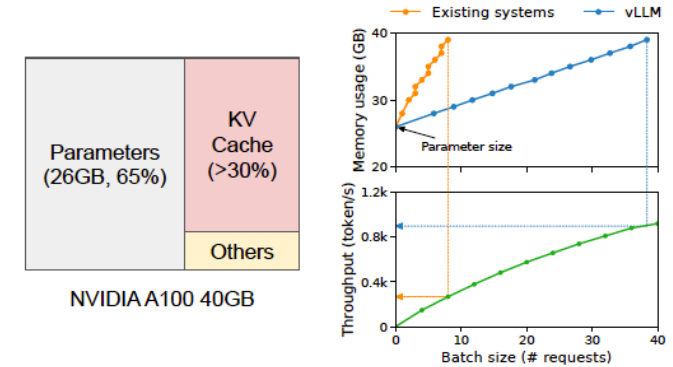


Figure 1. Left: Memory layout when serving an LLM with 13B parameters on NVIDIA A100. The parameters (gray) persist in GPU memory throughout serving. The memory for the KV cache (red) is (de)allocated per serving request. A small amount of memory (yellow) is used ephemeraly for activation. Right: vLLM smooths out the rapid growth curve of KV cache memory seen in existing systems [31, 60], leading to a notable boost in serving throughput.

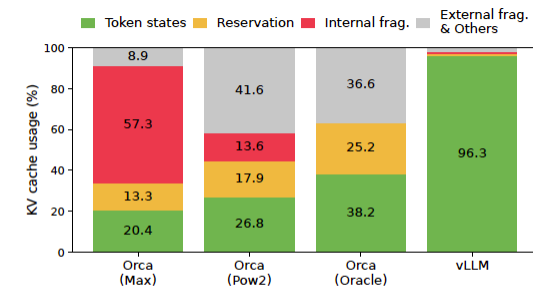


Figure 2. Average percentage of memory wastes in different LLM serving systems during the experiment in §6.2.

Paged Attention – System View

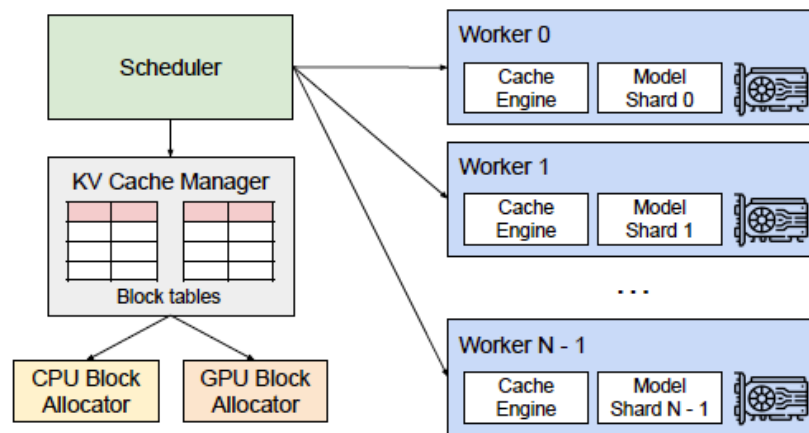


Figure 4. vLLM system overview.

- Each block in the GPU is allocated memory for fixed set of tokens (e.g. 16/32/64 etc.)
- Scheduler manages requests and the free memory pool of the GPU and maps incoming requests to available Blocks in the pool
 - Can be scaled to multiple worker nodes and GPU devices
 - Preemption, Swapping, Recomputation
- KV Cache manager, manages the Page Table, that is stored in the Host CPU, maps the logical KV blocks

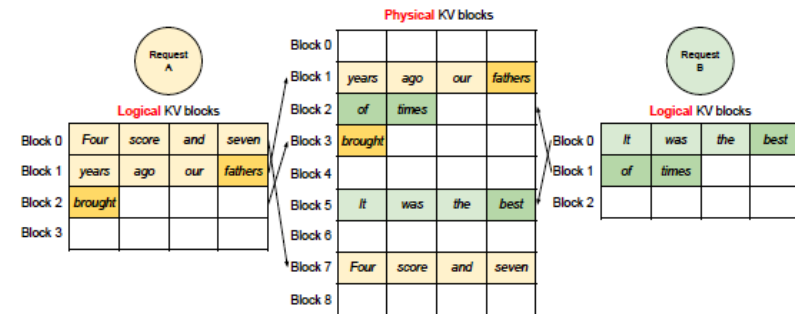


Figure 7. Storing the KV cache of two requests at the same time in vLLM.

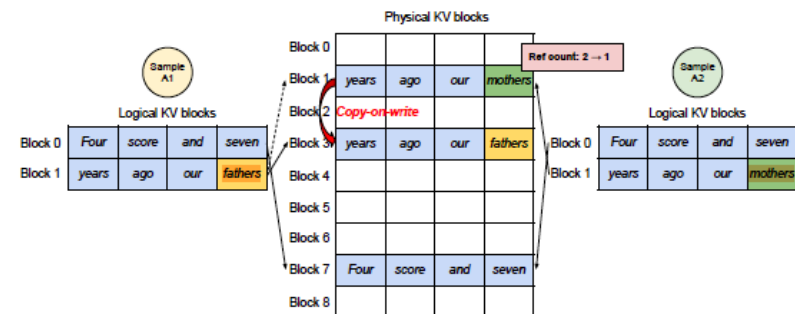
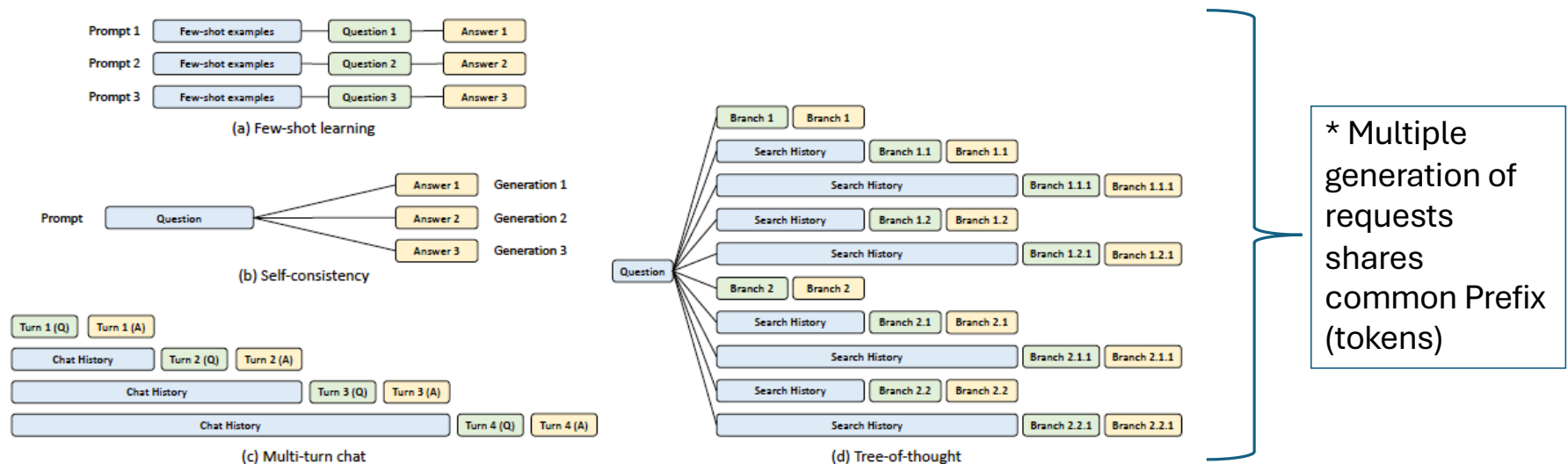


Figure 8. Parallel sampling example.

- Shows how the KV Caches can be used across multiple user requests using Prefix Caching and when the prompts are orthogonal

Radix Attention

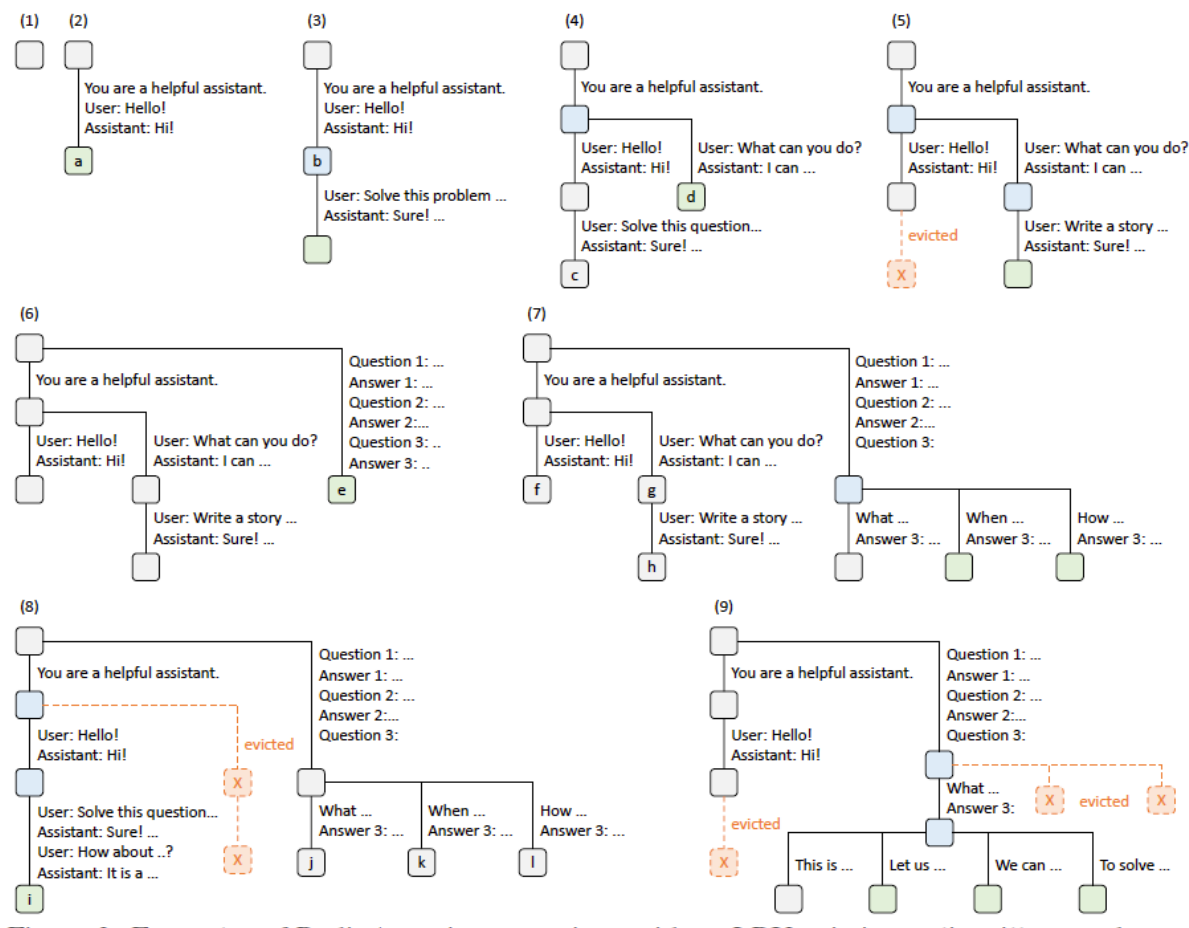
- Paged Attention Optimized for a single Prompt Request (KV cache discarded after a request is satisfied)
- Radix Attention extends this to optimize across multiple Prompt Requests (preserves KV Cache across user requests)
- Use cases – Few Shot Learning, Multi-Turn Chat, Tree-of-Thought, Multi-model (image & text)
- Paper report 6.4x higher throughput and Latency by 4x than vLLM



[1] Lianmin Zhang et al, “SGLang: Efficient Execution of Structured Language Model Programs”

Radix Tree Management

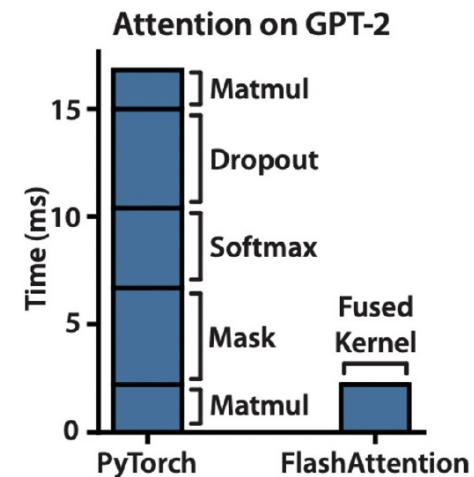
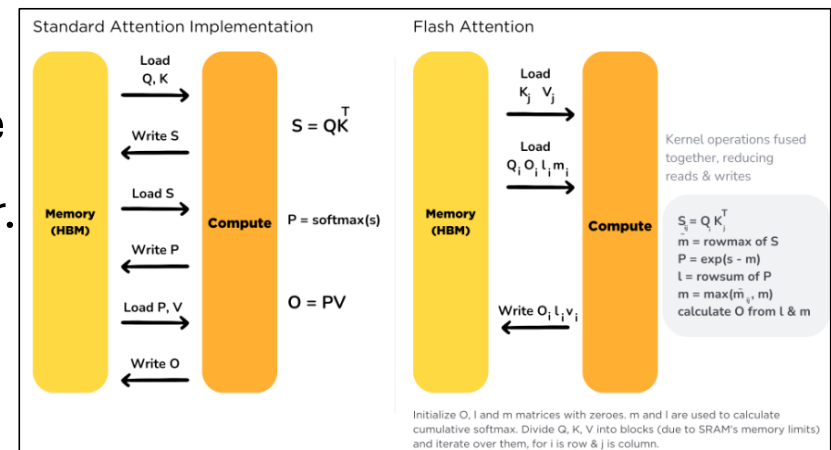
- Radix Tree manages the mapping between sequences of token and KV Cache Tensors
- This is memory intensive, hence techniques like LRU is used to evict least recently used leaf first
- Cached tokens and active requests share the same memory pool
- The Tree itself is stored in the CPU, a table maps the nodes to the GPU's memory block
- Cache aware scheduling is used to prefer longer prefix matched requests are prioritized to *maximize the cache hit ratio*.



Evolution of the Radix Tree

Flash Attention

- Flash Attention is an algorithm that invoked at the GPU's Kernel level to compute the Attention vectors in a memory and latency efficient manner.
 - Primarily trades-off Memory R/W with Compute
 - So highly memory efficient but compute intensive
- Optimization - Flash Attention [1] reorders the Q,K,V Matrix into Tiles that fit in a GPU SRAM (on-chip).
- It computes and applies the Softmax on the fly without storing the whole Q,K,V matrix
- Implemented as part of the Attention kernel where $Q \cdot K^T / V$ is applied.
- Improves Attention Latency (~6x) and lower training(~3x) time could be achieved
- The same authors also have proposed a improved FastAttention-2 version (2x speedup)



[1] Tri Dao et al, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness", NIPS 35, 2022, <https://arxiv.org/abs/2205.14135>

Flash Attention – Key Idea

- **Algorithm at a high level :**

- **Input :**

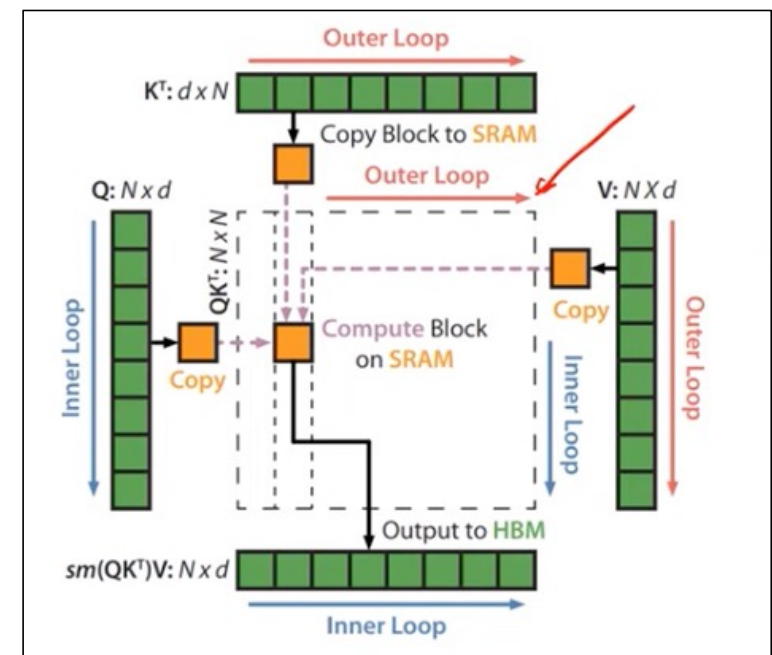
- The Q,K,V matrix each $N \times D$ is broken into smaller fixed size blocks

- **Outer loop:**

- Loop over blocks of K and V and load them into fast on-chip SRAM

- **Inner Loop:**

- Loop over Blocks of Q, load into the SRAM, and write the output of the Attention computation back to the HBM
 - For j^{th} (K,V), for i^{th} in Q, Compute K_j, V_j, Q_i in SRAM, and update O_i, l_i, m_i in HBM
 - O_i, l_i, m_i are temporary values that are read and written to, more efficient than working with the Q,K,V matrices.
 - Uses online Softmax algorithm with block matrix multiplication



Speculative Decoding

- **Decode generation is Memory bound operation, hence GPU is under utilized**
 - This approach is to increase the Arithmetic Intensity for a given inferencing system.
- **A draft model (smaller model) is used to generate future tokens, and then these tokens are appended to the prompt and made to verify by the larger GPU**
 - This would be equivalent of the Prefill, resulting higher GPU utilization, throughput (~2x)
 - Using Rejection Sampling algorithm to accept draft tokens, the vocabulary probability distribution is same as the large model (Fig 3)

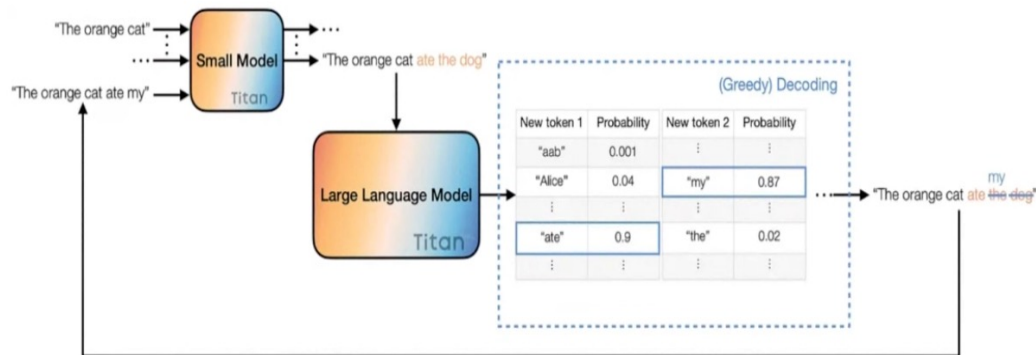


Fig: 1

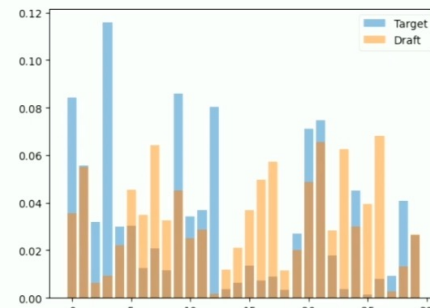


Fig: 2

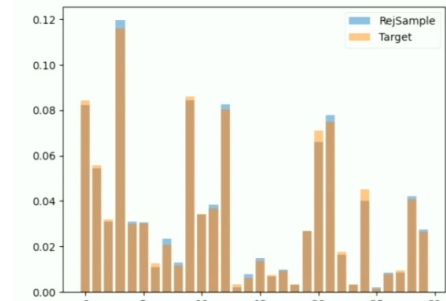


Fig: 3

[1] Charlie Chen et al, "**Accelerating Large Language Model Decoding with Speculative Sampling**",
<https://arxiv.org/abs/2302.01318>

Speculative Decoding

- **vLLM supports other other techniques instead of using a draft model**
 - *N-gram* – take the last *prompt_lookup_max* tokens; find a prior match in the sequence; if found, propose the k tokens that followed that match; otherwise decrement the window and retry down to *prompt_lookup_min* tokens
 - *Eagle* - perform "model surgery" on the large LM—keep embeddings and LM head, replace the transformer stack with a lightweight MLP; fine-tune that as a cheap draft
 - *Medusa* - train auxiliary linear heads on top (embeddings before LM head) of the large model to predict the next k tokens in parallel; use these heads to propose tokens more efficiently than running a separate small LM
- **Recent approaches also show how multiple draft models, even running on a CPU can increase throughput.**

Speculative Decoding: How to choose the draft model ?

- Objective – Draft must be fast and accurate enough so that the large model rarely rejects the tokens, and avoid duplication of compute
- **Smaller sibling model**
 - Use a smaller version from the same family (e.g. Llama-2-7B for Llama-2-70B)
 - Same Tokenizer , correlated predictions
 - Still requires 2 models
- **Distilled model**
 - Train small student to mimic large Teacher
 - Higher Alignment, fewer Mismatches
- **Quantized Copy**
 - Run a quantized (INT 8/4) version as a draft
 - Faster, similar predictions

Constraint Decoding

- Use cases – Output based on JSON format + Regex.
 - Earlier approach used JSON Schema -> Regular Expression -> FSM -> Logit Mask
- SGLang combined the logic of Decoding with the FSM required in the output

We can compress many deterministic paths in the state machine

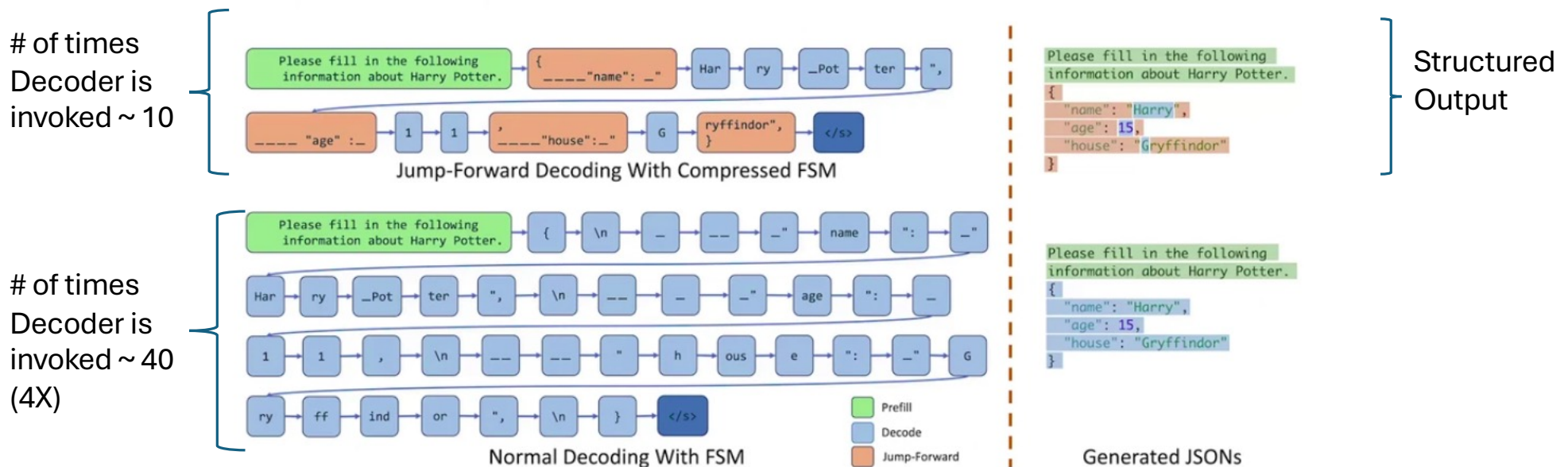


Diagram Ref: <https://huggingface.co/blog/paresh2806/sglang-efficient-llm-workflows>

P/D Disaggregation

- **Prefill and Decode has different characteristics**
 - Compute vs Memory Bound ; HOL Blocking issues (aggregated serving)
 - Main use case of this idea is Test-time Compute (e.g. long Context Windows, Reasoning and Planning, Ageantic workloads etc.)
 - Idea is to to physically decouple the Prefill computation from the Decode, scale independently and better interleaving of user request, possibly use different GPU types
 - Use different Parallelism techniques
 - Prefill prefers (TP and DP) for latency , while Decode prefers (PP) for Throughput
- **Its applicability depends on the distribution of the Input/Output Sequence Length (ISL/OSL)**
 - Adapting the P/D workers to dynamic changes in the ISL/OSL distribution is a consideration.
- **Also requires more intermediate functions to glue the two clusters (e.g. NIXL)**
- **Multi-layer Storage can be used for the KV Cache**

[1] Yinmin Zhong et al, “DistServe: Disaggregating Prefill and Decoding for Goodput-optimized LLM Serving”
<https://arxiv.org/pdf/2401.09670>

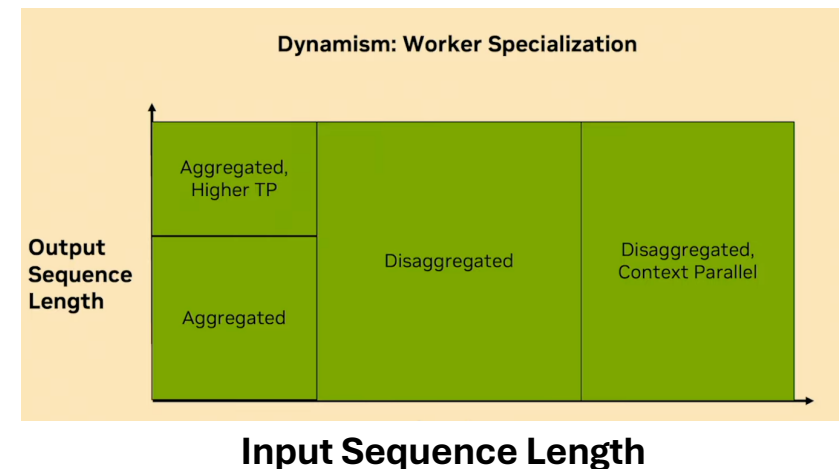
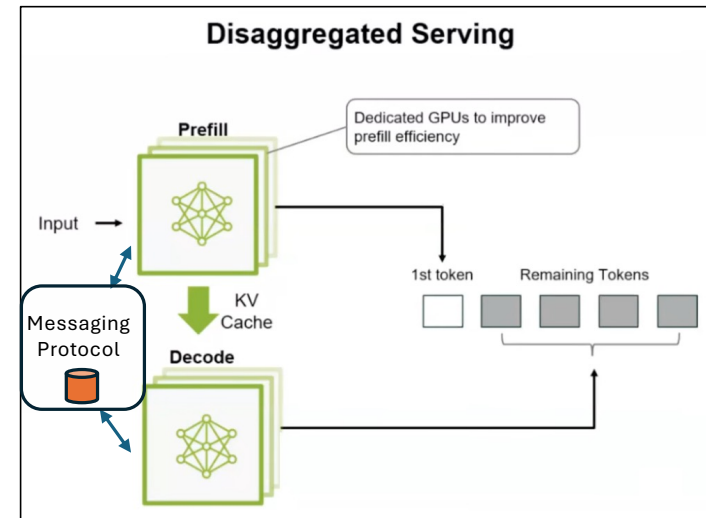
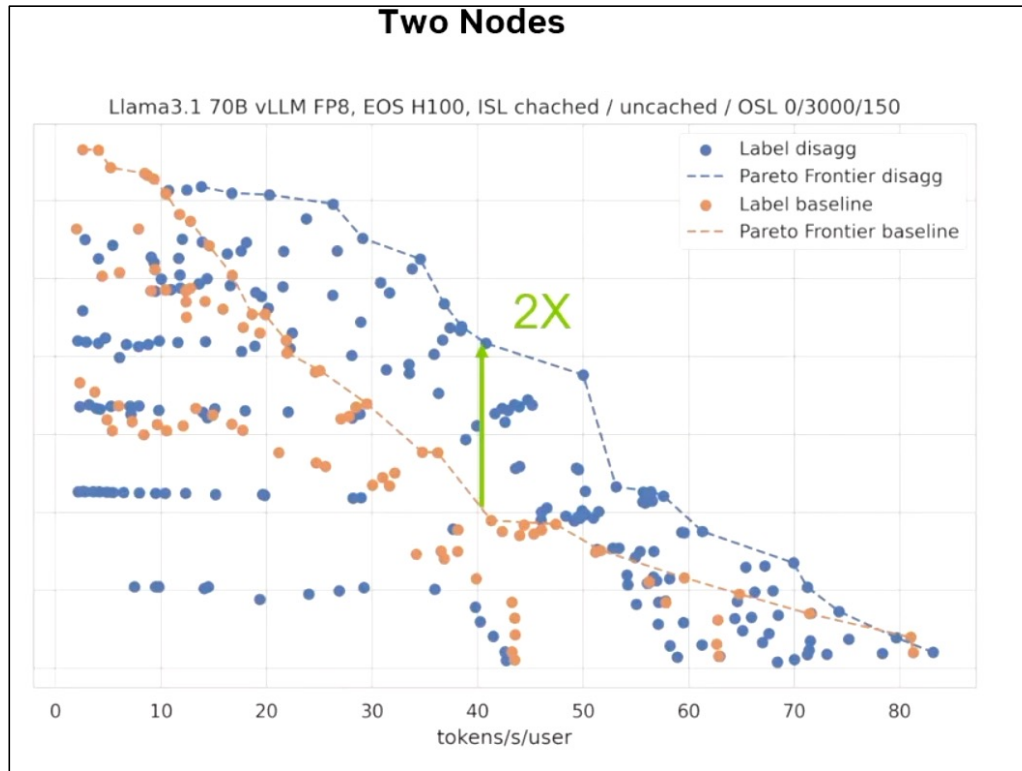


Fig : <https://www.youtube.com/watch?v=3C-6STonTLU>

P/D Disaggregation Performance

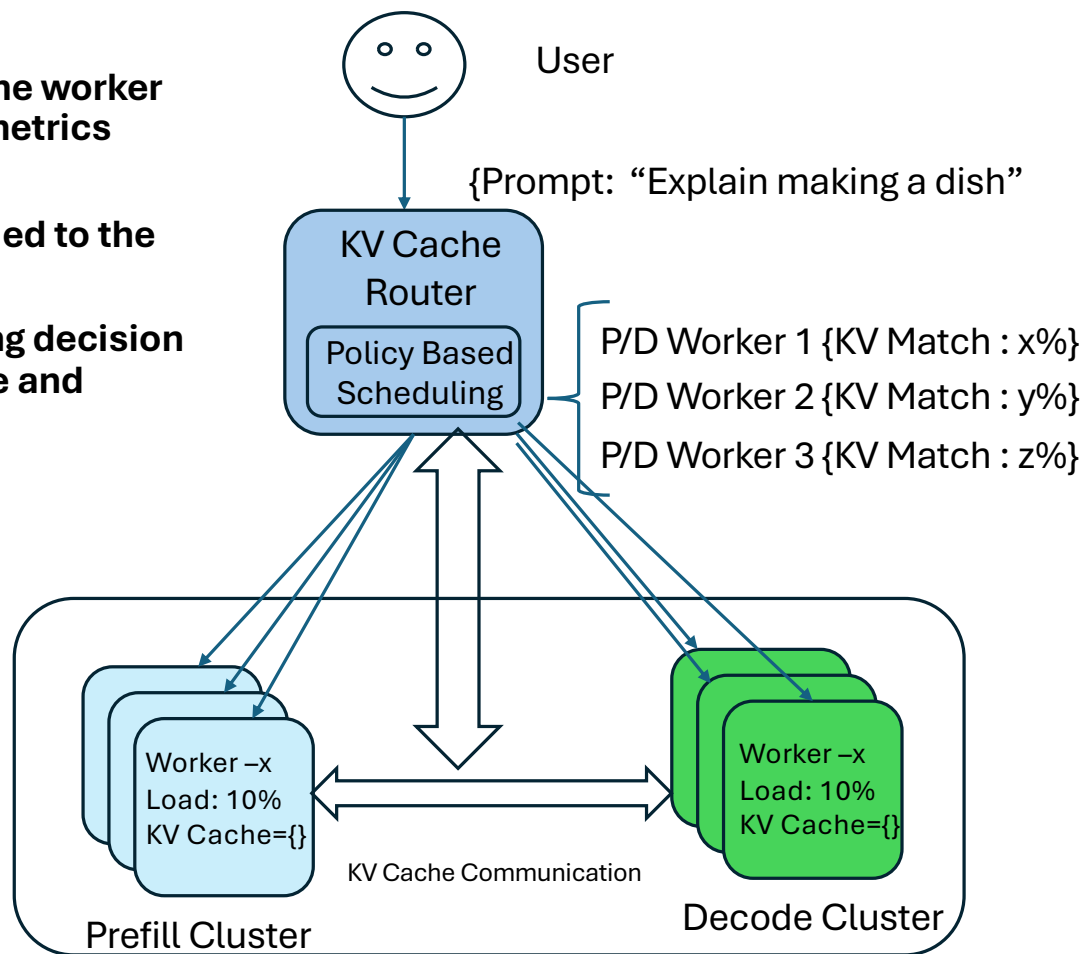
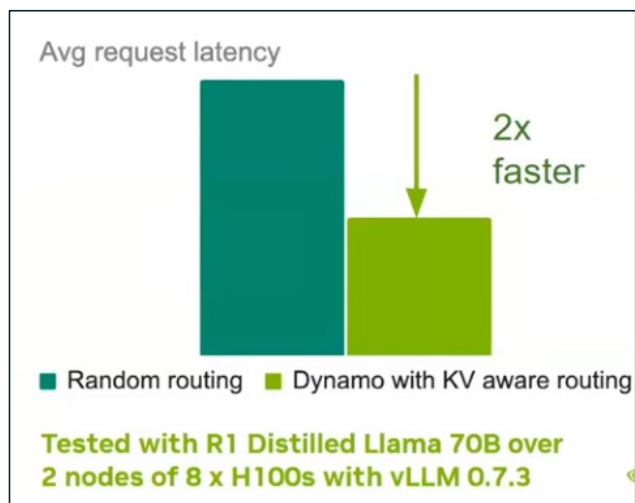


- This technique has been shown to result in 2X performance compared to aggregated serving

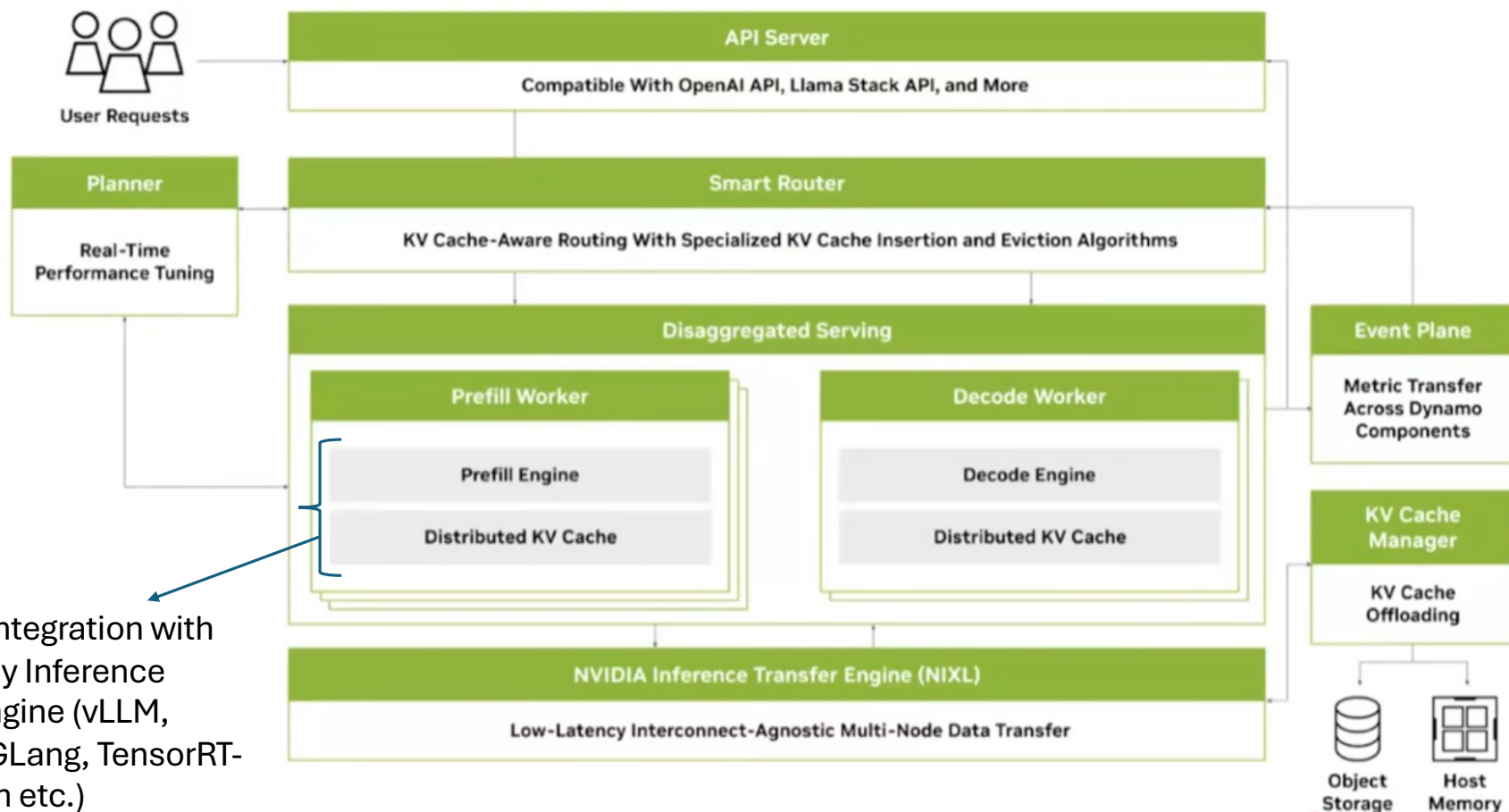
Source: Nvidia

KV Cache Routing

- KV Cache Router aware of the KV State in each of the worker nodes of the Prefill Cluster, along with other host metrics (CPU, Mem etc.)
- Prefill/Decode worker Prefix Cache statistics notified to the KV Cache Router
- KV Cache Router can make Policy Based Scheduling decision to choose the Prefill worker during the Prefill Phase and Decode Worker during the Decode Phase
- KV Aware Routing : 2x faster (Average Latency)



Nvidia's Dynamo P/D Disaggregated Architecture



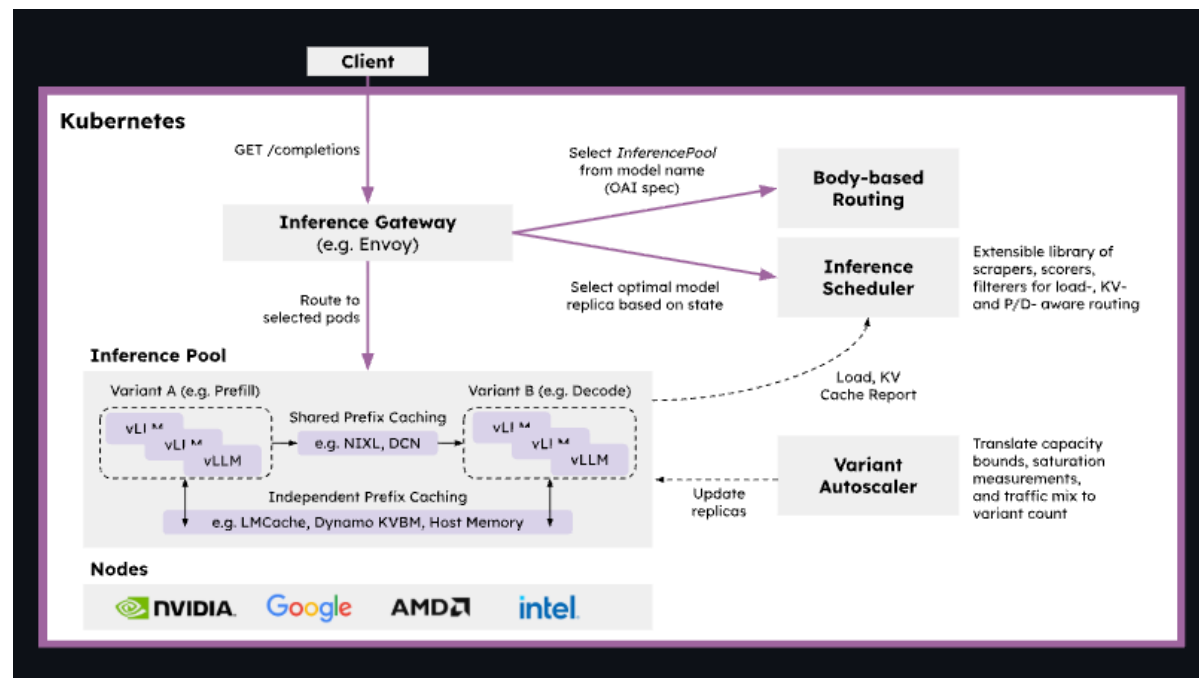
- Integration with any Inference Engine (vLLM, SGLang, TensorRT-llm etc.)

- Smart Router : Discovery, Request Routing, Events (Prefix Cache State), Data (NIXL) (ROCEv2/IB)

<https://github.com/ai-dynamo/dynamo>

Llm-d – Kubernetes Native Distributed Inferencing

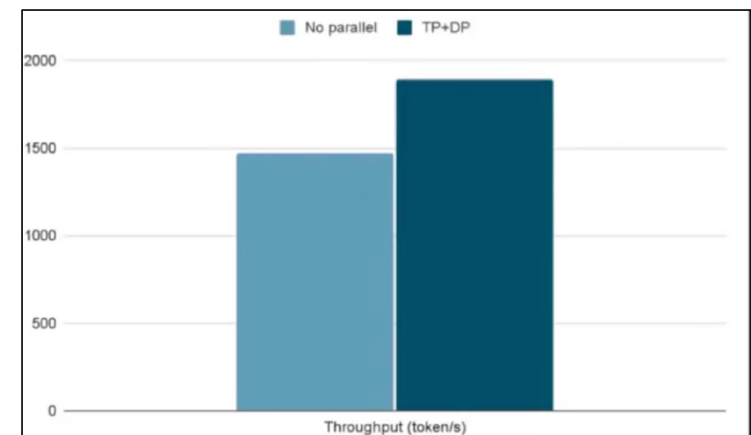
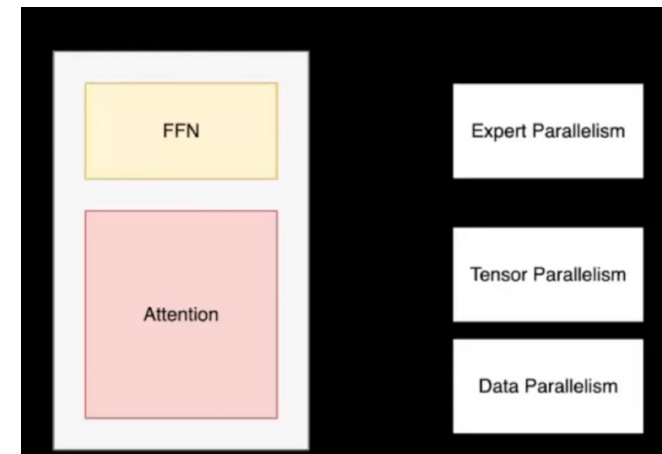
- KV Aware Routing, Scheduling, P/D functions implemented in Kubernetes distributions
- vLLM implements disaggregated prefilling by running two instances and handing off Prefix Caching, KV Caching distribution using LMCache and NIXL
- llm-d extends this with Kubernetes native orchestration for disaggregated serving and KV-aware Routing.
- Uses vLLM inference Engine



[1] <https://github.com/llm-d/llm-d>

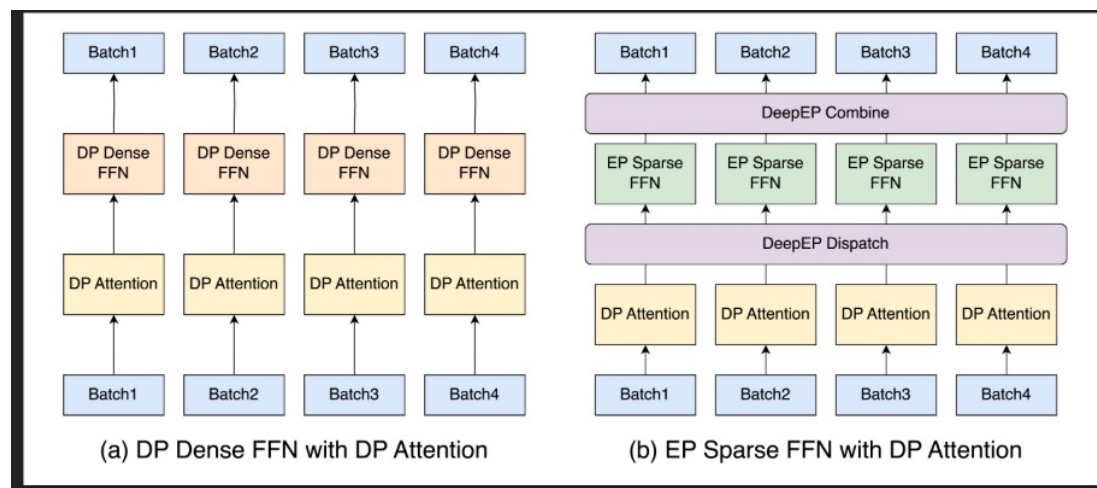
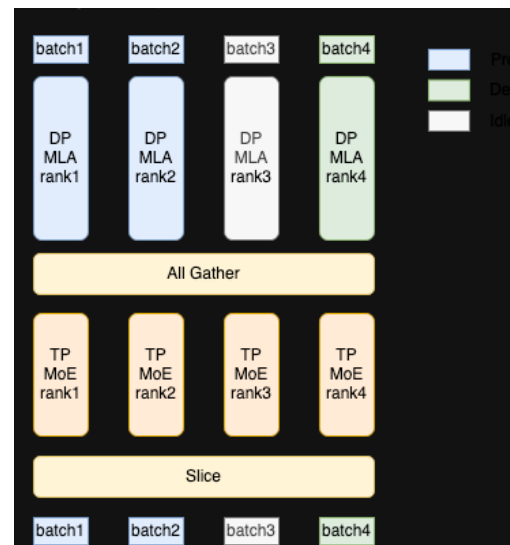
Multi-GPU Parallelism during Inferencing

- **Easy throughput gain during inferencing can be achieved by replicating the model on N GPUs**
 - If the model fits in a GPU
 - Sharding is required if the model doesn't fit in a GPU
- **Popular combination of parallelism used during inferencing**
 - Attention Layer : DP + TP
 - Sparse FFN/MOE : EP (Expert Parallelism) (tokens are distribute across multiple Experts where FFN is applied)
- **TP is usually used within the Server because of the high bandwidth inter-connect.**
- **But there is a trade-off when bigger batches are applied to a TP alone**
 - Graph shows the throughput gain when TP is used in combination with DP
 - For e.g. a 2 x 4 (DP X TP) means a two GPU clusters with 4 TP, serving multiple different batches of data
- **Optimization consideration here is on the overhead of communication (All-Reduce, Reduce Gather etc.), Batching**



SgLang - Large Scale Expert Parallelism

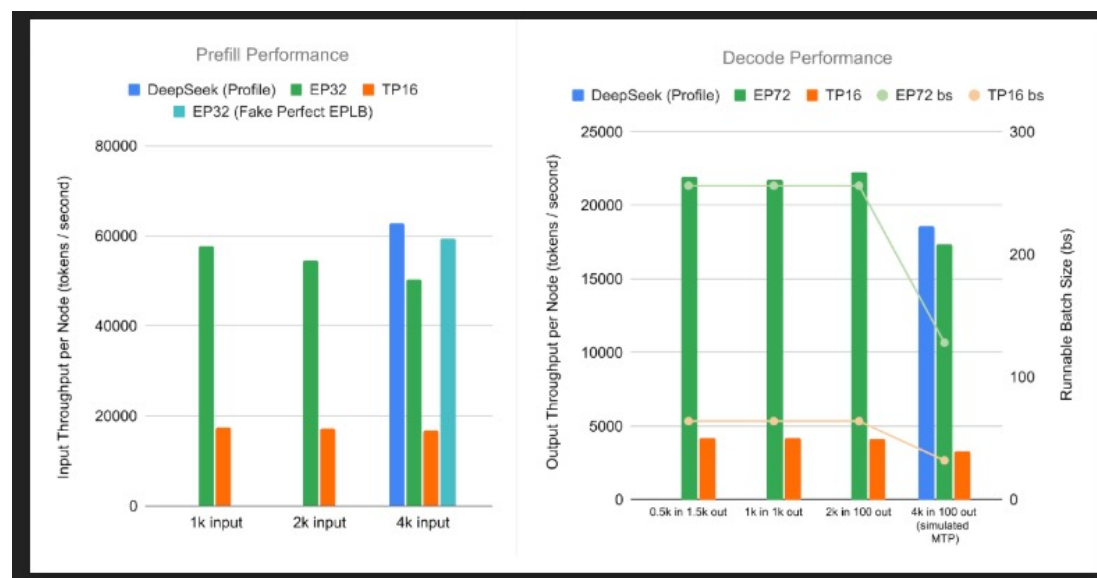
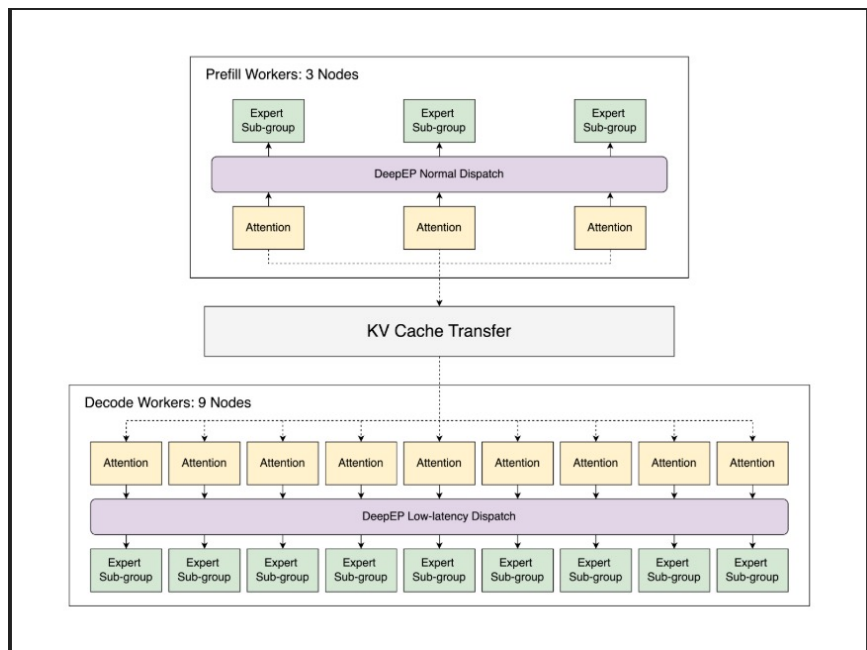
- This is an implementation in SGLang is to support DeepSeekV3 model which supports MOE and MLA
- Enables hybrid DP + TP Attention and only Dense FFN (Fig. a)
 - *Memory efficiency* - MLA uses a single KV Head, DP avoids replication across devices, hence splits tokens into sub-batches
 - *Communication overhead* :- DP Attention with low TP reduces per-device Memory, and reduces communication overhead (collective operations)
- Uses DeepEP Dispatch/Combine functions to enable Sparse FFN (MoE) model (Fig. b)
 - Scalability : Experts can be distributed across devices using EP, memory efficient
 - Load imbalance : EP introduces variability in routing, EPLB and DeepEP optimize for workload distributions



Ref : <https://lmsys.org/blog/2025-05-05-large-scale-ep/>

Performance with DeepEP

- Inference Throughput using SGLang using 12 Servers (8XH100) running DeepSeekv3
 - 24 GPU for Prefill and 72 for Decode
- P/D Disaggregation and full EP Optimization
- Independent evaluation of the Prefill and the Decode Phase
- Prefill Input Throughput ~ 55k Tokens/s, Decode Output Throughput ~ 22K Tokens/s



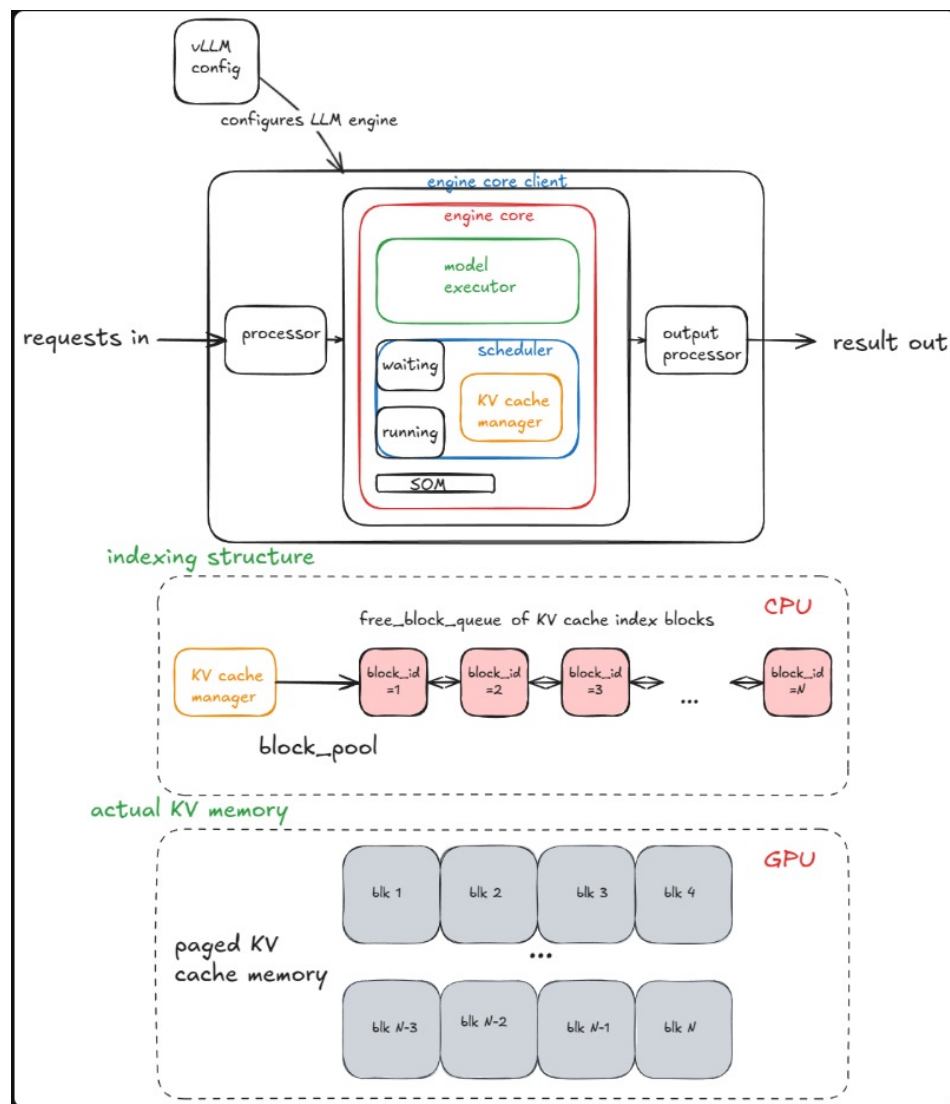
vLLM Architecture

- **Key Architecture functions:**

- In/Out Processor
 - Prompt Validation, (De)-Tokenization, Final output to the user
- Model Executor
 - Forward pass through Single/Multiple GPU Execution
- Scheduler
 - Waiting/Running Queues, Scheduling Policy (e.g. FCFS, Priority Queing)
 - KV Cache Manager

- **Optimization implementations**

- Paged Attention
- Chunked Prefill/Prefix Caching
- Guided Decoding
- Speculative decoding
- Disaggregated P/D
- Quantization



Ref: <https://www.aleksagordic.com/blog/vllm>

Structured Generation Language for LLM (SGLang)

• Architecture Functional Components

- Native Generation API (Native DSL for Prompt requests)
- BackEnd Engine
 - Tokenizer/Detokenizer
 - Model Worker
 - KV Cache Memory
 - Radix Tree
 - Scheduler
 - ..

• Optimization Features

- Radix Attention
- Continuous Batching
- Multi-Modal Support
- Structured/Constrained Output
- PD Disaggregation
- Distributed Inference
- Quantization

```
dimensions = ["Clarity", "Originality", "Evidence"]

@function
def multi_dimensional_judge(s, path, essay):
    s += system("Evaluate an essay about an image.")
    s += user(image(path) + "Essay:" + essay)
    s += assistant("Sure!")

    # Return directly if it is not related
    s += user("Is the essay related to the image?")
    s += assistant(select("related", choices=["yes", "no"]))
    if s["related"] == "no": return

    # Judge multiple dimensions in parallel
    forks = s.fork(len(dimensions))
    for f, dim in zip(forks, dimensions):
        f += user("Evaluate based on the following dimension:" +
            dim + ". End your judgment with the word 'END'")
        f += assistant("Judgment:" + gen("judgment", stop="END"))
    # Merge the judgments
```

Handle chat template and multi-modal inputs

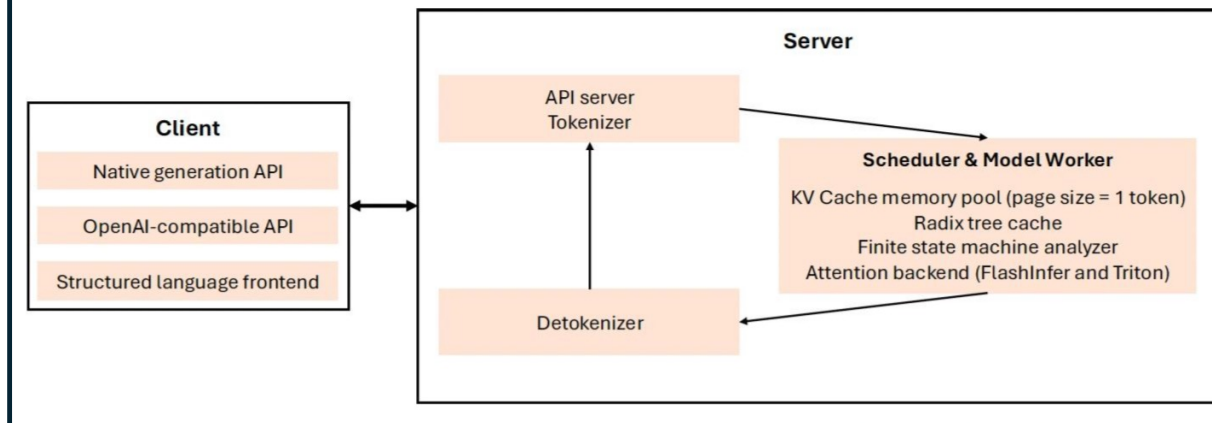
Select an option with the highest probability

Fetch result; Use Python control flow

Runtime optimization: KV Cache Reuse (Sec. 3)

Multiple generation calls run in parallel

SGLang architecture overview



<https://github.com/sgl-project/sglang>

Conclusions

- Inferencing is a multi-objective optimization problem that includes User SLO, Energy, Cost , Revenue etc.
- Current inferencing techniques focus on optimizing token throughput and latency.
- Techniques to optimize inferencing performance can be based on model or system-centric optimizations
- We surveyed many system-centric techniques that are used to increase throughput and reduce latency
- This area is an active area of research and with inferencing system requiring to serve multi-modal and ageantic capabilities, to serve diverse set of use cases for Telco, Enterprises, end-user and other vertical domains such as health or auto-motive

Thank You